Análisis Ejercicio 5 Programación Orientada a Objetos

1. **Herencia entre clases**

Clase	Relación	Datos propios	Función que implementa
Modulos	Clase padre	id, nombre, estadoSalud, activo, costoConstruccion	Define la información común y los métodos procesarCiclo y mostrarInformacion.
ModulosEnegia	Subclase que extiende Modulos	cantidadEnergia, energiaGenerada, cantidadCiclos	Aumenta la energía disponible de la misión cuando está activo.
ModulosVuelo	Subclase que extiende Modulos	tipoInstrumento, cantidadDatos, consumoEnergia, cantidadCiclos, totalHistorico	Consume energía y agrega datos en órbita.
ModuloTierra	Subclase que extiende Modulos	capacidadDescarga, consumoEnergia, estacion, totalDescargado	Descarga datos que están en órbita.

Los constructores de las tres clases hijas llaman a super(...) para inicializar los cinco atributos heredados. Esto evita declarar esos mismos atributos en cada tipo de módulo. La herencia también permite que App guarde objetos de los tres tipos en una única lista de Modulos.

Mision no hereda de Modulos. Su responsabilidad es mantener los valores de la misión: energía disponible, datos en órbita y datos descargados. Esta relación sirve para que los módulos reciban un objeto Mision para consultar o modificar esos valores.

2. Polimorfismo por herencia y por sobreescritura

Método	En Modulos	En las clases hijas
procesarCiclo(Mision mision)	Tiene un cuerpo vacío.	Cada tipo realiza su operación sobre la misión, según sus condiciones.
mostrarInformacion()	Devuelve los datos comunes del módulo.	Cada tipo añade sus datos particulares al texto obtenido con super.mostrarInformacion().

El polimorfismo se ve de mejor manera en el case 4 del menú, donde se procesan ciclos para todos los módulos de la misión:

```
case 4: //Se recorre toda la lista, por cada modulo se llama al procesar ciclo, como esta override, cada uno actua dife

    for(int i = 0; i < modulos.size(); i++)
    {
        modulos.get(i).procesarCiclo(mision);
    }

    mision.datosSiguienteCiclo(mision.getEnergiaDisponible(), mision.getDatosOrbita(), mision.getDatosDescargados());

    System.out.println(x: x: "Estado de la misión al terminar el ciclo:");
    System.out.println(mision);
    break;
```

Tipo de módulo	Condiciones que comprueba procesarCiclo	Cambio que realiza si puede operar
Energía	Que el módulo esté activo.	Suma cantidadEnergia a la energía de la misión; incrementa sus ciclos y su energía generada.
Vuelo	Que esté activo y que la misión tenga energía suficiente.	Suma cantidadDatos a los datos en órbita; resta energía; incrementa sus ciclos y su total histórico.
Tierra	Que esté activo, haya energía suficiente y existan datos en órbita.	Reduce los datos en órbita y aumenta el total descargado del módulo. La actualización de energía depende de la rama ejecutada, como se explica más adelante.

3. Responsabilidades de cada clase

Clase	Responsabilidades que tiene en el código
Mision	Guarda la identificación y operación de la misión, además de la energía y los datos. Ofrece métodos para modificar esos valores.
Modulos	Guarda los datos compartidos de los módulos y define los métodos que las clases hijas sobrescriben.
Clases hijas de Modulos	Guardan sus datos específicos y ejecutan la operación correspondiente a su tipo durante un ciclo.
App	Crea los objetos iniciales, mantiene la lista, presenta el menú, lee entradas, busca, ordena, modifica y agrega módulos, procesa ciclos y muestra el resumen de comunicaciones.

El programa puede utilizar MVC sin una interfaz gráfica porque la consola funciona como vista. En la implementación presentada, las clases Mision y Modulos con sus subclases representan el modelo; App contiene tanto la interacción por consola como el control del menú. Por ello, se identifican las funciones de MVC, aunque la vista y el controlador no están separados en clases distintas.